HiveMind: OS-Inspired Scheduling for Concurrent LLM Agent Workloads

Jay[1]

[1]Sperix Labs

April 16, 2026

Abstract

We present HiveMind, a scheduling system for concurrent LLM coding agents that applies operating system scheduling principles—admission control, fair queuing, backpressure, and token budget management—to eliminate the failure modes caused by uncoordinated parallel execution. When multiple LLM agents share rate-limited API endpoints, they exhibit resource contention patterns analogous to unscheduled OS processes competing for CPU, memory, and I/O. HiveMind sits as a transparent HTTP proxy between agents and API providers, requiring zero modifications to existing agent code. Our evaluation across 7 scenarios (5–50 concurrent agents) shows that uncoordinated agents fail at 72–100% rates under contention, while HiveMind reduces failures to 0–18% and eliminates 48–100% of wasted compute. An ablation study reveals that transparent retry—not admission control—is the single most critical scheduling primitive, but the primitives are most effective in combination. We formalize the OS–LLM agent scheduling analogy and provide an open-source implementation targeting Anthropic, OpenAI, and local model APIs.

1 Introduction

The emergence of AI coding agents—Claude Code, GitHub Copilot Workspace, Devin, SWE-Agent—has created a new class of compute workload: long-running, stateful processes that make repeated API calls to LLM providers. When users spawn multiple such agents in parallel, a natural pattern for tasks like generating test suites, writing proof-of-concept exploits, or refactoring across modules, the agents compete for shared resources:

- **API rate limits** (requests per minute, tokens per minute)
- **Network connections** (concurrent connection limits per endpoint)
- **Context windows** (fixed per model, cannot be shared)
- **API key quotas** (billing and access limits)

This resource contention leads to agent failures. In our motivating observation (§1.1), 3 out of 11 parallel agents died from `ECONNRESET` and HTTP 502 errors—a 27% failure rate—despite the API having sufficient aggregate capacity to serve all 11 sequentially.

1.1 Motivating Observation

On April 15, 2026, we spawned 11 concurrent Claude Code agents to generate proof-of-concept scripts for security findings. All 11 shared one Anthropic API key through a single network proxy. The results are shown in Table 1.

Table 1: Results of 11 uncoordinated concurrent agents

Outcome	Count	Percentage
Completed successfully	8	73%
Died (ECONNRESET)	2	18%
Died (HTTP 502)	1	9%

The 3 dead agents had each consumed approximately 45,000 tokens before failing—a total waste of ∼135,000 tokens and ∼15 minutes of wall time. The 8 surviving agents all completed successfully because they happened to stagger their requests enough to avoid the bottleneck.

Key insight: If the 11 agents had been staggered by just 5 seconds each, all 11 would have succeeded. The problem is not capacity—it is coordination.

1.2 Thesis

We propose that concurrent LLM agent workloads exhibit the same resource contention patterns as OS processes (CPU scheduling, memory pressure, I/O contention), and that applying OS scheduling principles—admission control, fair queuing, backpressure, and priority scheduling—to LLM agent orchestration eliminates the failure modes caused by uncoordinated parallel execution.

2 The OS Scheduling Analogy

We formalize the mapping between operating system concepts and LLM agent orchestration in Table 2. This analogy is not merely illustrative—it is structurally precise. Each OS mechanism addresses a specific resource contention failure mode that has a direct counterpart in the LLM agent domain.

Table 2: Mapping between OS and LLM agent scheduling concepts

OS Concept	HiveMind Equivalent	Analogy
Process	LLM agent	Stateful, long-running
CPU time	API request slots	Rate-limited, shared
Memory	Context window	Fixed per model
I/O bandwidth	Network connections	Limited concurrent
Scheduler	Admission + queue	Decides who runs next
Virtual memory	Checkpointing	Save state to disk
OOM killer	Token budget	Kill on budget exhaust
TCP congestion	AIMD backpressure	Latency-based adjust
Fork bomb protection	Max agents	Prevent runaway spawn
Nice levels	Task priority	User-specified priority

2.1 Why Existing Frameworks Fail

Current agent orchestration frameworks—LangChain, CrewAI, AutoGen, Semantic Kernel—treat the LLM API as an unlimited resource. They provide composition mechanisms (chains, crews, multi-agent conversations) but not resource management. Table 3 shows the gap.

They are, in OS terms, running a multi-process system without a scheduler.

Table 3: Scheduling capabilities of existing frameworks

System	Admission	Rate Limit	Backpressure	Budget	Priority
Claude Code (raw)	–	–	–	–	–
LangChain	–	Basic	–	–	–
CrewAI	–	–	–	–	Manual
AutoGen	–	–	–	–	–
Semantic Kernel	–	Basic	–	–	–
HiveMind	✓	✓	✓	✓	✓

3 Architecture

3.1 Transparent HTTP Proxy

HiveMind's key design decision is to implement scheduling as a transparent HTTP reverse proxy. Agents make normal API calls to `http://localhost:8765/v1/messages`, and HiveMind forwards them to the upstream provider after applying all scheduling logic.

```
Agent -> localhost:8765 -> HiveMind -> api.anthropic.com
```

This approach has critical advantages: (1) zero agent modification—works with any framework, SDK, or language; (2) provider agnostic—same proxy for Anthropic, OpenAI, Ollama; (3) observable—all traffic flows through one measurement point; (4) composable—can chain with other proxies.

3.2 Five Scheduling Primitives

3.2.1 Admission Control

A dynamic concurrency semaphore limits simultaneous in-flight API requests. The concurrency limit is initially configured (default: 5, based on observed Anthropic connection limits) and adjusted at runtime by the backpressure controller. Implementation: `asyncio.Semaphore` with dynamic resizing.

3.2.2 Rate Limit Tracking

Parses rate limit headers from API responses (`anthropic-ratelimit-requests-remaining`, `x-ratelimit-remaining-requests`, `retry-after`) and proactively pauses all agents before exceeding limits—preventing the burst of 429 errors that kills agents.

3.2.3 AIMD Backpressure

Additive Increase / Multiplicative Decrease, borrowed directly from TCP congestion control:

- If average latency $< L_{target}$: increase concurrency by α (additive increase)
- If average latency $> L_{target}$: multiply concurrency by β (multiplicative decrease)
- On error (429/502): immediately multiply concurrency by β

This creates a self-regulating system that automatically finds the optimal concurrency level.

3.2.4 Token Budget Management

Per-agent ceilings and a global token pool. At 85% usage, the agent receives a warning. At 100%, the agent is checkpointed (state saved to disk) and stopped.

3.2.5 Priority Queue with Dependency DAG

Tasks are ordered by: (1) priority level (CRITICAL > HIGH > NORMAL > LOW), (2) estimated complexity (shortest-job-first), (3) creation time (FIFO within same priority). Dependencies are tracked as a DAG with cycle detection.

3.3 Transparent Retry

The proxy intercepts 429, 502, 503, 529, and connection reset errors and retries transparently with exponential backoff plus jitter. The agent never sees the error—from its perspective, the request simply took longer.

3.4 Streaming Support

HiveMind passes through Server-Sent Events (SSE) streams without buffering, forwarding chunks as they arrive from the upstream API. Token counts are extracted from the `message_delta` and `message_start` events. Agents receive real-time streaming; HiveMind still counts everything.

4 Evaluation

4.1 Methodology

We evaluate HiveMind using a mock API server that simulates realistic Anthropic API behavior with configurable rate limits, error injection (random 502s, connection resets), rate limit headers, latency (base plus jitter plus spikes), and concurrency limits. Mock agents make N sequential API calls simulating multi-turn coding sessions. Each agent either completes all turns or "dies" on the first unrecoverable error, matching observed real-world behavior.

4.2 Scenarios

Table 4 shows our evaluation scenarios.

Table 4: Evaluation scenarios

Scenario	Agents	Rate Limit	Error Rate	Concurrency	Purpose
micro-5	5	50/min	0%	–	Baseline
micro-10	10	50/min	0%	–	Onset of contention
micro-20	20	50/min	0%	–	Significant contention
micro-50	50	50/min	0%	–	Extreme contention
replay-11	11	60/min	8% + 5%	5	Original failure case
stress	20	20/min	10% + 5%	3	Deliberate exhaustion
latency-spike	10	60/min	0%	–	AIMD test

4.3 Results

Table 5 presents the comparison between direct (uncoordinated) execution and HiveMind-managed execution.

At 5 agents, both modes succeed—there is no contention. At 10+ agents, uncoordinated execution fails catastrophically (72–100% failure rate), while HiveMind reduces failures to 0–18%.

Wall time trade-off. HiveMind takes longer because it serializes requests through the rate limit window rather than letting agents stampede and die. Direct mode "finishes fast"

Table 5: Direct vs HiveMind: failure rates and token waste

Scenario	Direct Fail%	HiveMind Fail%	Failure Δ (pp)	Waste Δ
micro-5	0.0	0.0	0.0	–
micro-10	100.0	10.0	−90.0	−100%
micro-20	100.0	10.0	−90.0	−94.4%
micro-50	100.0	0.0	−100.0	−100%
replay-11	72.7	18.2	−54.5	−48.3%
stress	100.0	10.0	−90.0	−100%
latency-spike	100.0	0.0	−100.0	−100%

only because agents fail immediately. When measured against *completed work*, HiveMind's throughput is vastly higher.

4.4 Ablation Study

To measure the individual contribution of each scheduling primitive, we run the replay-11 scenario with various primitives disabled (Table 6).

Table 6: Ablation study: contribution of individual primitives

Configuration	Alive	Dead	Fail%	Key Finding
Full HiveMind	11	0	0.0	Baseline
No admission	11	0	0.0	Rate limiter + retry compensate
No rate limiter	11	0	0.0	Admission + retry compensate
No backpressure	10	1	9.1	Marginal improvement
No retry	**4**	**7**	**63.6**	**Most critical primitive**
Admission only	**2**	**9**	**81.8**	**Insufficient alone**

Surprising finding: Our initial hypothesis was that admission control alone would be sufficient to fix the problem. The ablation disproves this—admission-only still produces 81.8% failure because it limits concurrency but does not handle rate limit errors or connection resets. **Transparent retry is the single most impactful primitive**, reducing failures from 63.6% (without it) to near-zero (with it). However, the primitives are most effective in combination: retry handles transient errors, admission prevents connection exhaustion, rate limiting prevents errors from occurring, and backpressure provides fine-grained stability.

5 Related Work

5.1 Agent Orchestration Frameworks

LangChain [1], CrewAI [2], AutoGen [3], and Semantic Kernel [4] focus on agent composition but not resource management. They assume the API is always available. HiveMind is complementary—it sits below any of these frameworks.

5.2 API Rate Limiting Libraries

Libraries like `tenacity` and `backoff` provide retry logic at the individual request level but lack system-wide coordination. Each agent retries independently, potentially amplifying load during rate limit windows (the "thundering herd" problem). HiveMind centralizes retry decisions across all agents sharing a key.

5.3 TCP Congestion Control

Our AIMD backpressure controller directly adapts the Additive Increase / Multiplicative Decrease algorithm from TCP Tahoe/Reno [5]. The key insight is that API latency serves the same role as network round-trip time: it signals congestion before packets (requests) are dropped.

5.4 OS Scheduling Theory

The correspondence between LLM agent scheduling and process scheduling has not been previously formalized in the literature. Classical scheduling theory—shortest-job-first [6], priority scheduling, multilevel feedback queues—applies directly when the "CPU" is an API request slot and "processes" are stateful agents with unpredictable execution times.

6 Limitations and Future Work

- **Single-machine.** Current implementation runs on one machine. Distributed scheduling across multiple machines sharing an API key is implemented via Redis but not yet evaluated at scale.
- **Token estimation.** Request token counting uses a heuristic (4 chars/token) when provider tokenizers are unavailable. Integration with provider-specific tokenizers is available as an optional dependency.
- **Dynamic priority.** Priority is set at submission time. Automatic priority adjustment based on observed progress is future work.
- **Real-world scale.** Our evaluation uses a mock API server. While it simulates realistic behavior (rate limits, errors, latency), production validation with actual API providers at scale is ongoing.

7 Conclusion

HiveMind demonstrates that OS scheduling principles are directly applicable to concurrent LLM agent workloads. The five scheduling primitives—admission control, rate limit tracking, AIMD backpressure, token budgets, and priority queuing—in combination eliminate the failure modes that currently plague parallel agent execution. The transparent proxy architecture requires zero changes to existing agents, making HiveMind a drop-in improvement for any multi-agent workflow.

Our ablation study yields a key insight for the field: in the current API landscape, *transparent centralized retry* is more important than *admission control* for agent survival, but both are most effective in combination. This suggests that LLM agent orchestration systems should prioritize retry coordination over simple concurrency limiting.

The system is open-source at https://github.com/sperixlabs/hivemind.

References

[1] Chase, H. (2022). LangChain. https://github.com/langchain-ai/langchain

[2] Moura, J. (2024). CrewAI. https://github.com/joaomdmoura/crewAI

[3] Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*

[4] Microsoft (2023). Semantic Kernel. https://github.com/microsoft/semantic-kernel

[5] Jacobson, V. (1988). Congestion Avoidance and Control. *ACM SIGCOMM Computer Communication Review*, 18(4), 314–329.

[6] Silberschatz, A., Galvin, P., & Gagne, G. (2018). *Operating System Concepts*, 10th ed. Wiley.